# Spec: Adaptive Stigmergic Memory Prototype

## Project Name

TasteMemory

## Build Target

Local-first prototype running on a Mac mini.

Initial domain: cooking and baking memory.

Long-term architecture: general adaptive memory substrate for AI agents.

## Goal

Build a local application that stores cooking/baking episodes as structured memories, retrieves them through different semantic lenses, and produces compact context packets at adjustable effort levels.

The core thesis is that AI performance can improve by increasing context density rather than only increasing model size. The system should retrieve, compress, and transform relevant memory so the main model sees the smallest useful context packet for the task.

## Strategic Framing

This is not a generic chatbot memory feature.

It is an adaptive memory/context engine with:

- structured episodic memory
- semantic memory
- procedural memory
- failure memory
- preference memory
- relation-weighted graph traversal
- effort-based retrieval
- memory lenses
- open-loop background questions
- reinforcement/decay mechanics
- context compilation for downstream agents

The system should eventually generalize from cooking/baking to lab automation, legal workflows, and other operational domains.

## Non-Goals for v0

Do not build:

- a full multi-agent swarm
- cloud deployment
- auth/multi-user permissions
- complex frontend
- production-grade vector database
- SaaS billing
- on-prem enterprise deployment
- external web browsing/inspiration stream
- autonomous task execution

Keep v0 local, inspectable, and easy to modify.

## Tech Stack

Preferred:

- Python 3.11+
- FastAPI backend
- SQLite for v0
- SQLAlchemy ORM
- ChromaDB or sqlite-vss for embeddings, whichever is simplest
- OpenAI API or local Ollama-compatible model abstraction
- Streamlit or simple React frontend
- Pydantic models
- pytest for tests

Acceptable simplification:

- Single FastAPI app
- SQLite only
- Embeddings stored as JSON/blob if vector search setup is annoying
- CLI first, UI second

## Local Directory Structure

```
taste_memory/
  README.md
  pyproject.toml
  .env.example

  app/
    main.py
```

```
  config.py

  db/
    base.py
    session.py
    models.py
    migrations/

  schemas/
    episode.py
    memory.py
    retrieval.py
    context_packet.py

  services/
    llm.py
    embeddings.py
    ingestion.py
    extraction.py
    retrieval.py
    lenses.py
    reinforcement.py
    context_compiler.py
    open_loops.py

  api/
    routes_episodes.py
    routes_memories.py
    routes_retrieval.py
    routes_open_loops.py

  prompts/
    extract_episode.md
    assign_lenses.md
    infer_relations.md
    compile_context.md
    skeptic_review.md

ui/
  streamlit_app.py

data/
  taste_memory.sqlite
  exports/
  seed/

tests/
  test_extraction.py
```

```
    test_retrieval.py
    test_context_compiler.py
```

## Core Concepts

### Episode

A concrete cooking/baking event.

Examples:

- Black Forest cake
- crème brûlée
- focaccia
- stuffed peppers
- ratatouille
- croquembouche attempt
- khachapuri

Episodes contain raw notes, ingredients, tools, constraints, actions, outcomes, and reflections.

### Memory

A durable unit extracted from an episode.

Memory types:

- episodic
- semantic
- procedural
- preference
- failure
- decision
- constraint
- workaround
- skill
- analogy
- open_loop

### Memory Lens

A retrieval frame that reorganizes memory by purpose.

Initial lenses:

- procedural
- workaround

- failure_mode
- constraint
- preference
- skill_building
- creativity
- analogy
- risk
- temporal
- source_authority

### Effort Level

A retrieval budget from 1–5.

Effort controls depth, creativity, and verification.

```
Effort 1:
  recent + obvious + high-confidence memories only

Effort 2:
  similar memories + key preferences + known constraints

Effort 3:
  graph relations + failure/workaround memories + skill progression

Effort 4:
  cross-lens retrieval + contradiction checks + abstract patterns

Effort 5:
  creative analogy search + open loops + deep synthesis + skeptic review
```

### Open Loop

A background unresolved question that should be triggered when new memories relate to it.

Example:

```
Question:
  Why do some ambitious bakes work while others become stressful?

Trigger patterns:
  time pressure, unfamiliar technique, missing equipment, many parallel
components, hard finishing step

When triggered:
```

```
  attach the new memory, update hypothesis, optionally propose a consolidated
principle.
```

### Memory Energy

Each memory and relation has an access cost.

Useful retrieval lowers future cost. Misleading retrieval raises future cost. Stale or unused memories decay.

This should be simple in v0.

## Data Model

### episodes

Fields:

```
id: string UUID
title: string
domain: string
date: datetime nullable
raw_notes: text
summary: text nullable
outcome_score: integer nullable 1-5
created_at: datetime
updated_at: datetime
```

### memories

Fields:

```
id: string UUID
episode_id: FK nullable
memory_type: string
content: text
abstract_pattern: text nullable
reason_type: string nullable
outcome: text nullable
confidence: float default 0.7
salience: float default 0.5
energy_cost: float default 1.0
decay_rate: float default 0.01
source: text nullable
created_at: datetime
```

```
updated_at: datetime
last_accessed_at: datetime nullable
access_count: integer default 0
success_count: integer default 0
failure_count: integer default 0
```

### entities

Fields:

```
id: string UUID
name: string
entity_type: string
canonical_name: string nullable
metadata_json: text/json
```

Entity types:

- ingredient
- dish
- tool
- person
- technique
- failure_mode
- constraint
- preference
- skill
- domain

### memory_entities

```
memory_id: FK
entity_id: FK
role: string nullable
```

### memory_relations

Fields:

```
id: string UUID
source_memory_id: FK
target_memory_id: FK
relation_type: string
strength: float default 0.5
```

```
energy_cost: float default 1.0
confidence: float default 0.7
success_count: integer default 0
failure_count: integer default 0
last_used_at: datetime nullable
created_at: datetime
```

Initial relation types:

```
caused_by
supports
contradicts
supersedes
similar_to
contrasts_with
worked_for
failed_for
balanced_by
requires
transfers_to
part_of
derived_from
```

### memory_lenses

```
memory_id: FK
lens: string
weight: float default 0.5
```

### open_loops

Fields:

```
id: string UUID
question: text
current_hypothesis: text nullable
trigger_patterns: text/json
priority: float default 0.5
energy_budget: integer default 3
status: string default active
created_at: datetime
updated_at: datetime
last_triggered_at: datetime nullable
```

### retrieval_events

Fields:

```
id: string UUID
query: text
effort_level: integer
lenses_used: text/json
memory_ids_returned: text/json
context_packet_id: string nullable
user_rating: integer nullable 1-5
notes: text nullable
created_at: datetime
```

### context_packets

Fields:

```
id: string UUID
query: text
effort_level: integer
compiled_context: text
memory_ids_used: text/json
warnings: text nullable
created_at: datetime
```

## Reason Types

Every action-like memory should classify why the action occurred.

Allowed reason types:

```
ideal_procedure
workaround
preference_choice
constraint_response
experiment
accident
error_recovery
unknown
```

This is critical because the system must distinguish:

- best practice
- forced workaround
- mistake
- personal preference
- one-off experiment

## API Endpoints

### Episodes

```
POST /episodes
GET /episodes
GET /episodes/{id}
PUT /episodes/{id}
DELETE /episodes/{id}
POST /episodes/{id}/extract
```

### Memories

```
GET /memories
GET /memories/{id}
PUT /memories/{id}
DELETE /memories/{id}
POST /memories/{id}/reinforce
POST /memories/{id}/penalize
```

### Retrieval

```
POST /retrieve
POST /compile-context
POST /rate-retrieval
```

### Open Loops

```
POST /open-loops
GET /open-loops
PUT /open-loops/{id}
POST /open-loops/check
```

## LLM Abstraction

Create a provider interface so models can be swapped.

```
class LLMProvider:
    def complete(self, prompt: str, system: str | None = None) -> str:
        ...

    def structured_complete(self, prompt: str, schema: type[BaseModel], system:
str | None = None) -> BaseModel:
        ...
```

Implement at least:

- OpenAI provider using environment variable `OPENAI_API_KEY`
- Dummy provider for tests

Optional:

- Ollama provider later

## Embeddings

Create an embedding abstraction.

```
class EmbeddingProvider:
    def embed(self, text: str) -> list[float]:
        ...
```

Store embeddings for:

- memory.content
- memory.abstract_pattern
- episode.summary

If vector DB setup takes too long, implement a placeholder semantic search using simple keyword scoring and leave a clear TODO.

## Extraction Pipeline

Input: raw episode notes.

Output:

- episode summary
- memory objects
- entities
- lenses
- relations

Pipeline steps:

```
1. Summarize episode.
2. Extract actions, constraints, outcomes, preferences, failures, lessons.
3. Classify memory types.
4. Classify reason types.
5. Extract entities.
6. Assign lenses.
7. Infer relations between new memories and existing memories.
8. Store all objects.
9. Generate embeddings.
```

## Extraction Prompt Requirements

The extraction prompt should ask the model to produce structured JSON.

Required extracted memory fields:

```
content
memory_type
reason_type
outcome
abstract_pattern
confidence
salience
entities
lenses
candidate_relations
```

Important instruction:

The model must distinguish between something done because it was ideal procedure and something done because of a constraint/workaround.

## Retrieval Algorithm v0

Inputs:

```
query
effort_level: 1-5
optional lenses
optional domain
```

Scoring formula:

```
score =
  semantic_similarity
  + salience
  + confidence
  + recency_bonus
  + lens_match_bonus
  + relation_strength_bonus
  + success_history_bonus
  - energy_cost
  - staleness_penalty
  - contradiction_penalty
```

Effort level modifies retrieval:

```
Effort 1:
  return top 5 direct memories
  no graph traversal
  no analogy lens unless explicitly requested

Effort 2:
  return top 8 direct memories
  include preferences and constraints

Effort 3:
  return top 12 memories
  include one-hop graph traversal
  include failures and workarounds

Effort 4:
  return top 16 memories
  include two-hop graph traversal
  include contradiction check
  include abstract patterns
```

```
Effort 5:
  return top 20 memories
  include analogy/creativity lenses
  include open-loop matches
  include skeptic review
```

## Context Compiler

Input:

```
query
retrieved memories
effort_level
lenses_used
```

Output:

A compact context packet.

Format:

```
Task:
  <user query>

Relevant current context:
  - ...

Relevant prior episodes:
  - ...

Constraints/preferences:
  - ...

Failure/workaround lessons:
  - ...

Transferable patterns:
  - ...

Open questions / risks:
  - ...

Suggested next direction:
```

```
  - ...

Sources:
  - memory IDs and episode titles
```

The compiler should prioritize dense, useful context over exhaustive detail.

## Reinforcement Mechanics v0

After a retrieval/context packet, allow user rating 1–5.

If rating >= 4:

```
for used memories:
  success_count += 1
  energy_cost *= 0.95
  salience += 0.03, max 1.0
  last_accessed_at = now

for used relations:
  success_count += 1
  energy_cost *= 0.95
  strength += 0.03, max 1.0
```

If rating <= 2:

```
for used memories:
  failure_count += 1
  energy_cost *= 1.05
  confidence -= 0.03, min 0.1

for used relations:
  failure_count += 1
  energy_cost *= 1.05
  strength -= 0.03, min 0.1
```

Do not reinforce merely because a memory was used. Reinforce only when the output was rated useful.

## Open Loop Mechanics v0

Open loops are checked during extraction and effort 5 retrieval.

A new memory triggers an open loop if:

```
semantic similarity to trigger patterns is high
OR entity overlap is high
OR lens overlap is high
```

When triggered:

- attach memory to open loop
- generate a short update
- optionally update current hypothesis

Do not interrupt normal retrieval unless effort level >= 4.

## UI Requirements

Build a minimal Streamlit UI.

Pages:

### 1. Add Episode

Fields:

- title
- domain
- date
- raw notes
- outcome score

Buttons:

- save
- extract memories

### 2. Browse Memories

Show table:

- content
- type
- reason type
- lenses
- salience
- confidence
- energy cost
- episode

Filters:

- memory type
- lens
- reason type
- episode

### 3. Retrieve

Inputs:

- query
- effort level slider 1–5
- lens multiselect
- domain optional

Output:

- retrieved memories
- compiled context packet
- warnings/risks
- rate result 1–5

### 4. Open Loops

Create and view open loops.

Show:

- question
- hypothesis
- trigger patterns
- last triggered
- status

## Seed Data

Include a seed script with at least 8 cooking/baking episodes based on manually entered notes.

Seed episodes:

1. Black Forest cake
2. crème brûlée
3. focaccia
4. khachapuri
5. stuffed peppers
6. ratatouille

7. croquembouche mini attempt
8. fruit bake in 8-inch round due to no muffin tin

Seed memories should include at least:

- bitter 78% ganache
- focaccia as repeatable win
- crème brûlée as repeatable win
- no mushrooms/raw onions/cilantro/goat cheese preference
- missing muffin tin constraint
- choux/pastry cream freezing issue
- stuffed pepper extra filling
- overdone khachapuri crust
- roasted tomato dip intensity issue
- ambitious multi-component bakes can become stressful

# Acceptance Criteria

## Functional

- User can add an episode.
- User can extract structured memories from raw notes.
- User can browse memories.
- User can retrieve memories with effort levels 1–5.
- Higher effort levels retrieve broader/deeper memories than lower levels.
- User can compile a context packet from retrieved memories.
- User can rate retrieval usefulness.
- Useful retrieval lowers energy cost.
- Poor retrieval raises energy cost.
- User can create open loops.
- Effort 5 retrieval checks open loops.

## Quality

- The system distinguishes procedure vs workaround.
- The system can answer:
- “What failures were caused by over-intensity?”
- “What did we only do because of missing equipment?”
- “What should I bake tonight based on prior wins and skill progression?”
- “What preferences should I remember when cooking for Devyn?”
- “What cooking lesson might transfer to lab automation?”
- Context packets should be concise and structured.
- Code should be modular enough to replace SQLite/vector search later.

### Tests

Write tests for:

- memory extraction schema validation
- retrieval scoring
- effort-level behavior
- reinforcement/penalty updates
- context packet generation
- open-loop trigger matching

## Example Query Behavior

Query:

```
What dessert should I make tonight?
```

Effort 1 should return:

- recent/repeatable wins
- obvious preferences
- simple recommendation

Effort 3 should add:

- skill progression
- recent failures
- constraints
- time/complexity considerations

Effort 5 should add:

- creative combinations
- analogy to prior successes
- open-loop about ambitious bakes
- skeptic warning about overcomplication

## Future Extensions

Do not build these in v0, but leave architecture room for:

- lab automation domain
- MCP server interface
- multi-agent roles
- external inspiration stream
- local LLM support through Ollama

- graph database
- web clipper
- source citations/files
- image ingestion
- scheduling background daemons
- role-specific agent temperaments
- permissioned memory scopes
- export/import memory graph

## README Requirements

README should include:

- project purpose
- setup instructions
- environment variables
- how to run backend
- how to run UI
- how to seed data
- example queries
- explanation of effort levels
- explanation of memory lenses
- explanation of reinforcement mechanics

## Implementation Order

1. Create project skeleton.
2. Define database models.
3. Add FastAPI routes.
4. Add LLM and embedding abstraction.
5. Implement episode creation.
6. Implement extraction pipeline.
7. Add seed data.
8. Implement retrieval scoring.
9. Implement context compiler.
10. Implement reinforcement.
11. Implement open loops.
12. Build Streamlit UI.
13. Add tests.
14. Write README.
15. Run the example queries and fix obvious issues.